End-to-End Cloud CI/CD Pipeline on Microsoft Azure

Complete Study & Revision Guide
Java web application • Jenkins • Gitleaks • SonarQube Cloud • Docker • ACR • AKS • Ansible

Item	Value
Repository	github.com/manisudhan/cicd-azure-cia2
Live endpoint	http://20.219.204.192
SonarCloud project	manisudhan_cicd-azure-cia2
Azure resource group	rg-cicd-cia2 (centralindia)
Container registry	acrcicdcia2.azurecr.io
Kubernetes cluster	aks-cicd-cia2 (1 x Standard_B2s, Kubernetes 1.36)
Jenkins	http://localhost:8090 (Docker container cicd-jenkins)

How to use this guide. Parts 1 and 2 give you the story and the justifications - that is what most viva questions are really testing. Part 3 is the rebuild procedure. Parts 4 to 6 are the reference detail. **Part 7 is the most important part for a live viva**: it is a set of recipes for the "change something right now" questions, such as adding a pod or editing the pod spec. Part 10 is a question bank.

Contents

Part	Title	What it covers
1	The Problem and the Solution	What was broken, the architecture, the end-to-end flow
2	Why Each Tool	Justification for every choice, and the alternatives rejected
3	Building It From Scratch	The manual procedure, in order, with commands
4	Jenkins In Detail	Image, plugins, credentials, the job, and exactly when it triggers
5	The Pipeline, Stage By Stage	All 8 stages: purpose, code, failure modes
6	Kubernetes Deep Dive	Every manifest and every field, explained
7	Live Operations Recipes	The "do this now" questions: add a pod, change the spec, roll back
8	Security	What is protected, how, and the honest limitations
9	Problems Actually Hit	Real failures from this build and their root causes
10	Question Bank	~45 likely viva questions with answers
A	Command Cheat Sheet	Everything you might need to type under pressure
B	File Inventory	What each file in the repository is for
C	Numbers To Memorise	The handful of facts worth knowing cold

Part 1 - The Problem and the Solution

1.1 The problem, stated properly

A team builds a Java web application by hand. A developer compiles it locally, runs the tests when they remember to, builds a Docker image with an ad-hoc tag, and copies it to a server. If you are asked "what was wrong with the old process?", do not answer "it was manual". Name the four specific failures:

1. **Not reproducible.** The artifact depends on whichever JDK, Maven version and local configuration that developer happened to have. "It works on my machine" cannot be disproved.
2. **Error-prone.** Any step can be skipped. Tests get bypassed under deadline pressure and an untested build reaches users.
3. **Serialising.** One person owns the deploy, so releases queue behind that person's availability.
4. **Untraceable.** With tags like `latest` or `v2-final`, nobody can say which commit is running, which makes rollback guesswork.

Every design decision later in this project answers one of those four. That mapping is the single most useful thing to have in your head.

1.2 Architecture

```
Developer
   | git push
   v
GitHub (public repo)  <----- polled every 2 min -----+
                                                     |
                                          Jenkins (local Docker)
                                                     |
   +-------------------------------------------------+
   |  Declarative pipeline, 8 stages
   v
1 Checkout        git metadata -> IMAGE_TAG = <build>-<commit>
2 Gitleaks        secret scan over FULL git history     [security gate]
3 Maven           compile, 29 unit tests, JaCoCo coverage
4 SonarQube Cloud static analysis + Quality Gate wait   [quality gate]
5 Docker build    multi-stage image, tagged <build>-<commit>
6 Push to ACR     acrcicdcia2.azurecr.io
7 Deploy          Ansible renders manifests, kubectl apply
8 Verify          rollout status + HTTP smoke test on the public IP
   |
   +--> Azure Container Registry --(AcrPull via managed identity)--> AKS
                                                                      |
                                                  Azure Load Balancer (public IP)
                                                                      |
                                                                  Browser
```

Trust boundaries. GitHub and SonarQube Cloud are external SaaS. ACR and AKS live inside one Azure resource group. Jenkins runs on the developer's own machine and authenticates to Azure with a service principal scoped to that resource group only.

1.3 The whole flow in one paragraph

A developer pushes to `main`. Within two minutes Jenkins notices the new commit by polling, and starts a build. It scans the entire git history for credentials, compiles the code, runs the unit tests, measures coverage, sends the result to SonarQube Cloud and waits for the Quality Gate verdict. If all of that passes it builds a container image tagged with the build number and the commit SHA, pushes it to Azure Container Registry, then has Ansible render the Kubernetes manifests for that exact image and apply them to AKS. Kubernetes performs a rolling update in which a new pod must pass its readiness probe before an old one is removed. Finally the pipeline waits for the Azure Load Balancer's public IP and makes an HTTP request to it, asserting that the live application

reports the image tag this very build produced. Nobody touched anything.

The sentence that wins marks. "The pipeline does not just deploy - it proves the deployment. Stage 8 queries the live endpoint and fails the build if the running application does not report this build's image tag. Without that, a green pipeline only means *kubectl apply* succeeded, which it does even when the pods then crash-loop."

Part 2 - Why Each Tool

Expect "why did you use X?" for every single tool. A good answer has three parts: what problem it solves, why it beat the alternative, and what it costs you.

2.1 Git and GitHub

Source-code management, required by the brief. The repository is **public** for three concrete reasons: the brief asks for instructor-accessible code; SonarQube Cloud is free only for public projects; and GitHub webhooks and polling both work without extra credentials. The cost is that nothing secret can ever live in the repository - which is exactly why Gitleaks is in the pipeline.

2.2 Maven, via the Maven Wrapper

Maven is the standard Java build tool: a declarative `pom.xml` describes dependencies and the build lifecycle, so the build is the same everywhere. The important detail is the **wrapper** (`./mvnw`), which is committed to the repository and pins Maven 3.9.9. The build agent therefore needs no Maven installed at all. That directly removes failure 1 (not reproducible): there is no "which Maven version is on the server?" question.

Lifecycle phases you should be able to name

Phase	What it does
validate	Checks the project is correct and all information is available
compile	Compiles source into target/classes
test	Runs unit tests with Surefire; does not package
package	Builds the JAR/WAR into target/
verify	Runs checks on the package - this is where the JaCoCo report is generated
install	Copies the artifact into the local ~/.m2 repository
deploy	Publishes the artifact to a remote repository

The pipeline runs `clean verify`, not `package`. Why? Because the JaCoCo report is bound to the `verify` phase, and SonarQube needs that report for coverage. Running `package` would silently give Sonar zero coverage.

2.3 JUnit 5 and JaCoCo

JUnit 5 with AssertJ for assertions and MockMvc for the HTTP layer. 29 tests cover the service logic, the configuration binding and both controllers without starting a real server. JaCoCo instruments the bytecode at test time and produces the XML report that SonarQube consumes.

Counter	Covered	Missed	Coverage
Instruction	290	13	95.7%
Branch	23	3	88.5%
Line	70	5	93.3%
Method	18	1	94.7%
Class	8	0	100%

2.4 Gitleaks - the security gate

The problem it solves. Automating a pipeline means artifacts get published faster. If a developer commits an API key, automation publishes that key faster too. A secret that reaches a registry or a public repository must be treated as compromised and rotated - you cannot un-publish it.

Why Gitleaks specifically. It scans the full git *history*, not just the working tree. That matters enormously: the usual pattern is that someone commits a credential, notices, deletes it in the next commit, and believes the problem is gone. The secret is still in the object database and still recoverable by anyone who clones the repository. `gitleaks git .` walks every commit, so it finds it. It is also a single static binary with roughly 170 built-in rules, which makes it trivial to put in a container image.

Why it is stage 2, right after checkout. It costs about a second and it must run before anything is published. Putting it after the Docker build would mean a leaked credential could already be inside an image layer.

What was added. Six custom rules for the credential types this project actually handles, because the default rule set does not know about them:

- `azure-storage-connection-string` - connection strings containing AccountKey
- `azure-service-principal-secret` - clientSecret / appSecret values
- `kubeconfig-embedded-client-key` - base64 client keys in a kubeconfig
- `acr-admin-password` - registry admin passwords
- `sonar-token` - sq[apu]_ prefixed tokens
- `jenkins-api-token` - Jenkins user API tokens

The allowlist. Narrow and documented: this config file itself (it contains the detection patterns), the Maven wrapper scripts, build output, screenshots, and obvious placeholder patterns such as `CHANGE-ME`. The principle is to allowlist specific known-safe paths rather than weaken a rule.

Prove the gate is not a no-op. A planted fake Sonar token and a fake Azure client secret produced `leaks found: 3`; a clean tree reports `no leaks found`. If you are asked "how do you know it works?", that is the answer: it was tested in both directions. A security control you have only ever seen pass is a control you have not tested.

2.5 SonarQube Cloud - the quality gate

The problem it solves. Same asymmetry as Gitleaks. Shipping faster is only an improvement if what you ship is not worse. Static analysis finds bugs, vulnerabilities and code smells that tests do not: a null dereference on an untested branch, a resource that is never closed, a hard-coded credential pattern, a cognitive-complexity hotspot.

Why it is a gate rather than a report. This is the key point. The pipeline runs it with `-Dsonar.qualitygate.wait=true`, which blocks the build until SonarQube Cloud has finished computing the Quality Gate, and fails the build if the gate is red. A dashboard that nobody looks at changes nothing; a gate that fails the build cannot be ignored. *"The difference between a gate and a dashboard is whether it can stop you."*

What the Quality Gate checks (the default "Sonar way", applied to new code): no new blocker/critical issues, no new security hotspots unreviewed, coverage on new code at or above 80 percent, duplicated lines on new code below 3 percent. This project reports 95.7 percent instruction coverage, comfortably above the threshold.

Why the Cloud version rather than self-hosted. Self-hosted SonarQube needs about 3 GB RAM, its own database, patching and backups - operational burden with no educational return here. SonarQube Cloud is free for public repositories and removes all of it. The trade-off: your code is analysed by a third party, and you depend on their availability.

The trap. SonarQube Cloud enables **Automatic Analysis** by default on GitHub-imported projects, and Automatic Analysis is mutually exclusive with CI-based analysis. If it is left on, the pipeline's analysis is rejected with "You are running CI analysis while Automatic Analysis is enabled". Fix: project -> Administration -> Analysis Method -> turn Automatic Analysis off. It can also be done through the API: `POST /api/autoscan/activation` with `enable=false`.

2.6 Docker, and why multi-stage

Containerisation makes the artifact self-contained: the image carries the JRE, the application and nothing else, so it runs identically on a laptop and on AKS. That is the direct answer to failure 1.

Multi-stage build. Stage 1 uses `maven:3.9.9-eclipse-temurin-21` and carries the full JDK plus Maven - roughly 600 MB of tooling. Stage 2 starts again from `eclipse-temurin:21-jre-alpine` and copies in only the built JAR. The published image is around 190 MB instead of 600 MB. Two benefits: faster pulls, and a much smaller attack surface, because a compiler and a build tool that are not in the image cannot carry a CVE or be used by an attacker.

Why dependencies are a separate layer. The Dockerfile copies `pom.xml` and runs `mvn dependency:go-offline` *before* copying `src`. Docker caches layers, so a source-only change reuses the cached dependency layer instead of re-downloading the whole repository. Order your Dockerfile from least-frequently-changing to most-frequently-changing.

Why tests are skipped inside the Dockerfile. They already ran in stage 3, where Jenkins can collect the JUnit XML and publish the trend chart. Running them again inside the image build would double the time and hide the results.

2.7 Azure Container Registry

A private registry in the same cloud as the cluster. The decisive feature is that AKS can authenticate to it with the cluster's **managed identity** - `az aks update --attach-acr` grants the kubelet identity the `AcrPull` role. That means **no registry password exists anywhere**: not in Kubernetes as an imagePullSecret, not in Jenkins, not in a manifest. That is a genuine security improvement over Docker Hub with a stored credential.

2.8 Azure Kubernetes Service

A managed Kubernetes control plane. Why Kubernetes rather than just running a container on a VM: declarative desired state, self-healing (a crashed container is restarted, a failed pod rescheduled), rolling updates with health gating, and horizontal scaling. Why AKS rather than self-managed Kubernetes: the control plane is free on the Free tier and Microsoft operates it, the Azure Load Balancer integration turns `type: LoadBalancer` into a real public IP automatically, and `metrics-server` is installed by default so the HorizontalPodAutoscaler works out of the box.

2.9 Jenkins, and why not GitHub Actions

Jenkins was chosen deliberately over the easier option. A self-hosted controller shows the whole DevOps stack instead of hiding it behind a managed runner: you have to provision the agent, install the toolchain, manage credentials and secure the server. That is also what gives Ansible real work to do. And the `Jenkinsfile` lives in the repository, so the pipeline is version-controlled alongside the code and changes to it go through review like any other change.

Why it runs in a local Docker container rather than on an Azure VM. A VM for the controller cost roughly US$38/month (the B2s instance, a second static public IP and a second managed disk) for something only needed while someone is actively working. Moving it into a container removed that entirely and cut the Azure bill from about US$4.50/day to about US$2.05/day with no loss of pipeline function. The cost of that decision: GitHub cannot reach `localhost`, so the automatic trigger is SCM polling rather than a webhook.

Option	Advantage	Why not chosen
GitHub Actions	No server at all, free minutes	Hides the build infrastructure; leaves no real role for Ansible
Azure DevOps Pipelines	Azure-native	Free tier needs a parallelism grant that can take days
Jenkins on an Azure VM	Real public webhook endpoint	About US$38/month for a controller used a few hours a day
Jenkins in local Docker	Free, reproducible via Dockerfile	No inbound webhook; polling instead (chosen)

2.10 Ansible

Configuration management and deployment automation, agentless - it needs only SSH, or nothing at all when it runs against localhost. Two playbooks:

- `deploy-app.yml` **- used by the pipeline.** Stage 7 calls it. It verifies kubectl can reach the cluster, renders the manifests for this build's image tag, applies them, waits for the rollout and reports the endpoint. This is deployment automation.
- `jenkins-setup.yml` **- the alternative topology.** Turns a bare Ubuntu VM into an identical build server: Java, Jenkins, Docker, kubectl, Azure CLI, Gitleaks, Ansible. Kept because it documents the cloud-hosted option; not used because Jenkins runs locally. This is configuration management.

Why Ansible rather than a shell script. Idempotence. An Ansible task describes the desired state ("this package is present", "this service is started"), so re-running the playbook after a change does only the work that is still needed. A shell script describes actions and generally cannot be re-run safely.

Part 3 - Building It From Scratch

The manual procedure, in dependency order. Roughly 60-90 minutes, most of it waiting for Azure.

Step 0 - Prerequisites

Tool	Check	Why it is needed
Git	git --version	Source control; also used by Gitleaks
Docker Desktop	docker info	Runs Jenkins, and builds the application image
Azure CLI	az version	Creates and talks to Azure resources
GitHub CLI	gh --version	Creating and pushing the repository (optional)
Java 21	java -version	Only to run the app outside Docker

Maven is **not** required: the wrapper is committed. Ansible is **not** required locally: it is inside the Jenkins image. Docker Desktop must use the **WSL 2 backend**, because the Jenkins container mounts the host Docker socket.

Step 1 - Repository

```
gh auth login
gh repo create cicd-azure-cia2 --public --source=. --remote=origin --push
```

Public, so SonarQube Cloud analysis is free and the instructor can read it.

Step 2 - Azure infrastructure

```
az login
bash infra/azure-setup.sh
```

The script does five things, and the order matters:

1. **Resource group** `rg-cicd-cia2`. One group means one cost boundary and one-command teardown.
2. **Register resource providers.** A new or newly renewed subscription has namespaces such as `Microsoft.ContainerRegistry` unregistered, and resource creation then fails with `MissingSubscriptionRegistration` - a message that looks nothing like the real cause.
3. **Container registry** `acrcicdcia2`, Basic SKU. The name must be globally unique across all of Azure.
4. **AKS cluster** `aks-cicd-cia2`: 1 node, `Standard_B2s`, Free tier control plane, `--attach-acr` to grant AcrPull, `--node-osdisk-size 32` because the 128 GB Premium SSD default costs about US$20/month and is billed even while the cluster is stopped.
5. **Service principal** `sp-cicd-cia2` with three role assignments: Contributor on the resource group, AcrPush on the registry, and Azure Kubernetes Service Cluster Admin on the cluster.

> The service principal password is displayed **once**. The script also writes it to `sp-credentials.json`, which is git-ignored, so that the credential-loading script can read it without anything being retyped.

Step 3 - Start Jenkins

```
docker compose up -d --build
```

The first build takes about five minutes: it installs the Docker CLI, Azure CLI, kubectl, Gitleaks and Ansible into the image and pre-installs the plugins listed in `jenkins/plugins.txt`. Jenkins is then on `http://localhost:8090`. Port 8090 rather than 8080 because a separately installed native Jenkins may already hold 8080.

The setup wizard is skipped entirely: `-Djenkins.install.runSetupWizard=false` plus `jenkins/init.groovy.d/01-security.groovy`, which creates the admin user from `JENKINS_ADMIN_USER` and `JENKINS_ADMIN_PASSWORD` supplied by the git-ignored `.env` file.

Step 4 - SonarQube Cloud

1. Sign in at sonarcloud.io **with GitHub**.
2. Create a free organisation and import the repository.
3. **Turn Automatic Analysis off** (Administration -> Analysis Method). This is the step people miss.
4. Note the organisation key and project key.
5. Generate a token: avatar -> My Account -> Security.
6. Put the two keys into the `Jenkinsfile` (`SONAR_ORG`, `SONAR_PROJECT_KEY`), commit and push.

Step 5 - Credentials into Jenkins

```
SONAR_TOKEN=<token> bash infra/jenkins-azure-credentials.sh
```

Reads `sp-credentials.json` and creates all four credentials through the Jenkins script console, so nothing is typed into a web form. Or do it by hand: Manage Jenkins -> Credentials -> System -> Global credentials.

Step 6 - The job

```
bash infra/jenkins-bootstrap.sh --build
```

Creates the job from `jenkins/job-config.xml` via the REST API and starts a build. By hand: New Item -> Pipeline -> Build Triggers: Poll SCM -> Pipeline script from SCM -> Git -> repository URL -> branch `*/main` -> Script Path `Jenkinsfile`.

Step 7 - Prove a change flows through

```
# edit the release message
sed -i 's/APP_RELEASE_MESSAGE: .*/APP_RELEASE_MESSAGE: "Release 2"/' k8s/01-configmap.yaml
git commit -am "Demo: change the release message"
git push
# within 2 minutes Jenkins starts a build by itself; then refresh the browser
```

Step 8 - Protect the credits

```
bash infra/azure-pause.sh     # ~0.35 USD/day instead of ~2.05
bash infra/azure-resume.sh    # bring it back
bash infra/azure-teardown.sh  # delete everything
```

Part 4 - Jenkins In Detail

4.1 What is in the image, and why

`jenkins/Dockerfile` extends `jenkins/jenkins:2.516.3-lts-jdk21` and adds every tool the `Jenkinsfile` shells out to. The point is that the build agent is **defined as code**: there is no "someone installed kubectl on the server once" step that cannot be reproduced.

Tool	Which stage needs it
docker CLI	Stage 5 builds the image. Only the CLI - the daemon is the host's, reached through the mounted socket
az (Azure CLI)	Stages 6 and 7: authenticate, az acr login, az aks get-credentials
kubectl	Stages 7 and 8: apply manifests, rollout status, smoke test
gitleaks	Stage 2 secret scanning
ansible	Stage 7 deployment automation
envsubst (gettext-base)	k8s/render.sh substitutes the image tag into the manifests
git, python3, jq	Checkout metadata and small parsing tasks

No Docker-in-Docker. The container mounts `/var/run/docker.sock` and uses the *host's* daemon. There is no nested daemon and no `--privileged`. The cost is that the container runs as root to open that socket, which is root-equivalent on the host - listed honestly as a limitation in Part 8.

4.2 Plugins, and what each is for

Plugin	Purpose
workflow-aggregator	The Pipeline suite - without it a Jenkinsfile cannot run at all
pipeline-stage-view	The stage-by-stage graphical view; this is what you screenshot
git	Cloning the repository and SCM polling
github	GitHub integration and the /github-webhook/ endpoint
credentials-binding	Provides withCredentials, which injects secrets as env vars and masks them in logs
plain-credentials	The Secret text credential type
junit	Parses target/surefire-reports/*.xml into the test trend chart
timestamper	Timestamps on every console line - very useful when diagnosing a slow stage
ws-cleanup	Workspace cleanup between builds

4.3 Credentials - the four, and how they are used

ID	Kind	Contents	Used by
azure-sp	Username/Password	Service principal appId and password	Stages 6 and 7, for az login
azure-tenant-id	Secret text	Entra ID tenant GUID	Stages 6 and 7, az login --tenant
azure-subscription-id	Secret text	Subscription GUID	Stages 6 and 7, az account set
sonarcloud-token	Secret text	SonarQube Cloud analysis token	Stage 4, -Dsonar.token

They are injected only inside a `withCredentials` block, which binds them to environment variables for the duration of that block and **masks them in the console log** - you see `-Dsonar.token=****` and a line saying "Masking supported pattern matches of $SONAR_TOKEN". Outside the block the variables do not exist.

Why a service principal and not a password or a personal account

A service principal is a non-human identity for an application. It can be scoped narrowly, revoked without affecting a person, and rotated independently. This one has exactly three role assignments and nothing subscription-wide.

4.4 When does the pipeline trigger? (read this twice)

Three ways a build starts. Be precise about which one you are demonstrating.

Trigger	Declared as	When it fires
SCM polling	pollSCM('H/2 * * * *')	Jenkins asks GitHub every ~2 min whether main has a new commit; if yes, it builds. This is the primary automatic trigger here
GitHub webhook	githubPush()	GitHub POSTs to /github-webhook/ the instant you push. Needs Jenkins reachable from the internet - it is not, on localhost, unless you run a tunnel
Manual	Build / Build with Parameters	A human clicks, or something POSTs to the REST API

Reading the cron field

Jenkins cron has five fields: minute, hour, day-of-month, month, day-of-week. `H/2 * * * *` means "every 2 minutes". The `H` is Jenkins-specific and stands for **hash**: instead of every job firing exactly on the minute, Jenkins spreads load by hashing the job name to pick an offset. `H/2` is therefore "every two minutes, at a consistent but arbitrary offset". Writing `*/2` would make every job on the controller poll simultaneously.

Why polling rather than a webhook here

Jenkins is on `localhost`, which GitHub cannot reach - inbound connections to a laptop are not routable. Polling works because it is **outbound**: Jenkins asks GitHub, not the other way round. This still satisfies "the pipeline runs automatically", because no human clicks Build. The cost is up to two minutes of latency and a small amount of wasted polling traffic. A webhook is instant and quieter, which is why production systems prefer it.

If you want an instant webhook for a demo, expose Jenkins with a free tunnel: `cloudflared tunnel --url http://localhost:8090`, then set the webhook Payload URL to `https://<id>.trycloudflare.com/github-webhook/`. The URL changes each restart.

4.5 Pipeline options, and what each prevents

Option	Effect
timestamps()	Prefixes every console line with a timestamp
buildDiscarder(logRotator(numToKeepStr: '20'))	Keeps only the last 20 builds, so the disk does not fill
timeout(time: 30, unit: 'MINUTES')	Aborts a hung build instead of occupying the executor forever
disableConcurrentBuilds()	Queues builds instead of running two at once. Essential here: two builds deploying to the same namespace would race

4.6 Parameters

Parameter	Values	Effect
DEPLOY_TARGET	azure (default), local	azure pushes to ACR and deploys to AKS. local deploys to Docker Desktop Kubernetes and skips stage 6. Stages 1-5 are identical
RUN_SONAR	true (default), false	Untick to prove the rest of the pipeline before SonarCloud is configured

Why azure is the default. A build started by SCM polling uses the defaults. If the default were `local`, a plain `git push` would deploy to the laptop rather than to Azure, and the "change reaches the public endpoint automatically" demonstration would not work.

Part 5 - The Pipeline, Stage By Stage

For each stage: what it does, why it exists, and what makes it fail. Stages run in order and the build stops at the first failure, so a defect never reaches the cluster.

Stage 1 - Checkout

Clones the commit, then reads the short SHA, author and subject with `git rev-parse` and `git log -1`, and derives the single most important variable in the pipeline:

```
env.IMAGE_TAG  = "${env.BUILD_NUMBER}-${env.COMMIT_SHA}"      // e.g. 8-95e0d05
env.FULL_IMAGE = "${ACR_LOGIN_SERVER}/${IMAGE_NAME}:${IMAGE_TAG}"
```

Why that tag format. The build number orders releases; the commit SHA ties the image to exact source. Given a running pod you can name the commit and the Jenkins build that produced it. That is the answer to failure 4 (untraceable).

Why not `latest`? Because the pod template would be byte-identical between builds, so `kubectl apply` reports "unchanged", Kubernetes does nothing, and the pipeline goes green while the old code keeps serving. This is the single most common broken-CD bug. A unique tag per build guarantees the pod spec changes, which guarantees a real rolling update.

Stage 2 - Secret Scan (Gitleaks)

```
gitleaks git . --config .gitleaks.toml --redact \
    --report-format sarif --report-path gitleaks-report.sarif \
    --exit-code 1 --verbose
```

- `git .` - scan the whole history, not just the working tree
- `--redact` - do not print the secret itself into the build log, which would simply move the leak
- `--exit-code 1` - fail the build on any finding
- SARIF report archived as a build artifact for the audit trail

Fails when: any committed credential matches a rule, anywhere in history.

Stage 3 - Build and Unit Test (Maven)

```
./mvnw -B -ntp clean verify
```

`-B` is batch mode (no interactive prompts, cleaner logs); `-ntp` suppresses transfer progress spam. The `post { always { ... } }` block publishes JUnit results and archives the JAR and the JaCoCo report **even when the stage fails** - which is exactly when you most want to see which test broke.

Fails when: compilation fails, or any of the 29 tests fails.

Stage 4 - Code Quality (SonarQube Cloud)

```
./mvnw -B -ntp sonar:sonar \
    -Dsonar.host.url=https://sonarcloud.io \
    -Dsonar.organization=manisudhan \
    -Dsonar.projectKey=manisudhan_cicd-azure-cia2 \
    -Dsonar.token=$SONAR_TOKEN \
    -Dsonar.qualitygate.wait=true -Dsonar.qualitygate.timeout=300
```

Skipped when RUN_SONAR is false, via `when { expression { return params.RUN_SONAR } }`.

Fails when: the Quality Gate is red; the token is wrong; or Automatic Analysis is still enabled on the project.

Stage 5 - Docker Build and Tag

```
docker build \
    --build-arg IMAGE_TAG=${IMAGE_TAG} --build-arg COMMIT_SHA=${COMMIT_SHA} \
```

```
    --build-arg BUILD_NUMBER=${BUILD_NUMBER} \
    -t ${FULL_IMAGE} -t ${LATEST_IMAGE} .
```

Two tags: the immutable `<build>-<commit>` used for deployment, and `latest` as a convenience pointer. The build args become OCI labels and environment variables inside the image, so the provenance travels with the artifact.

Fails when: the Dockerfile is broken, or the Docker daemon is unreachable.

Stage 6 - Push to Azure Container Registry

```
az login --service-principal -u $AZ_CLIENT_ID -p $AZ_CLIENT_SECRET -t $AZ_TENANT_ID
az account set --subscription $AZ_SUBSCRIPTION_ID
az acr login --name acrcicdcia2
docker push ${FULL_IMAGE}
docker push ${LATEST_IMAGE}
az acr repository show-tags --name acrcicdcia2 --repository cicd-demo-app \
    --orderby time_desc --top 5 --output table
```

Only runs when `DEPLOY_TARGET == 'azure'`. The service principal holds `AcrPush`, not registry ownership. The final command prints the tag list as in-log evidence.

Fails when: the SP lacks AcrPush, or the registry name does not match.

Stage 7 - Deploy to Kubernetes

```
az aks get-credentials -g rg-cicd-cia2 -n aks-cicd-cia2 --admin --overwrite-existing
ansible-playbook ansible/deploy-app.yml -i ansible/inventory.ini \
    -e deploy_target=azure -e image=${FULL_IMAGE} -e image_tag=${IMAGE_TAG} \
    -e build_number=${BUILD_NUMBER} -e commit_sha=${COMMIT_SHA} \
    -e k8s_namespace=cicd-demo
```

The playbook checks connectivity, runs `k8s/render.sh` (which uses `envsubst` on a named allow-list of variables, so nothing else in the YAML is touched), applies the rendered manifests, waits for the rollout and reports the endpoint. The rendered YAML is archived as a build artifact - so you can always see exactly what was sent to the cluster.

Fails when: the SP cannot fetch cluster credentials (needs the Cluster Admin role), or the rollout does not complete in 300 s.

Stage 8 - Verify Deployment

```
kubectl -n cicd-demo rollout status deployment/cicd-demo-app --timeout=300s
kubectl -n cicd-demo get deployment,pods,svc -o wide
# wait for the LoadBalancer public IP, then:
curl -fsS http://$PUBLIC_IP/actuator/health
DEPLOYED=$(curl -fsS http://$PUBLIC_IP/api/v1/info)
echo "$DEPLOYED" | grep -q "${IMAGE_TAG}" || exit 1
```

This is the stage that makes the pipeline trustworthy. It does three escalating checks: the rollout completed (so new pods passed readiness); the service answers HTTP; and the running application reports *this build's* image tag. The last assertion makes a false success impossible - a green build cannot mean "apply succeeded but the old version is serving".

Fails when: pods crash-loop, no public IP is assigned within 5 minutes, or the live endpoint reports a different image tag.

post blocks - what runs regardless

Block	Action
success	Logs the image and target cluster
failure	Dumps pod status and the deployment description - diagnostics captured at the moment of failure

Block	Action
always	az logout, so no Azure session is left behind in the container

Failure containment summary

Failure	Caught by	Consequence
Committed API key	Stage 2	No image built or published
Broken unit test	Stage 3	JUnit report shows which test
New bug or coverage drop	Stage 4	Quality Gate red, build fails
Bad Dockerfile	Stage 5	Nothing pushed
Wrong Azure permissions	Stage 6	Cluster untouched, old version still serving
Pod crashes on startup	Stage 8	rollout times out; maxUnavailable: 0 means the old ReplicaSet served throughout

Part 6 - Kubernetes Deep Dive

Five manifests in `k8s/`. Be able to explain any field.

6.1 Core objects, in one table

Object	What it is
Pod	The smallest deployable unit: one or more containers sharing a network namespace and storage. You rarely create one directly
ReplicaSet	Keeps N identical pods running. Created and managed by a Deployment - one per pod-template version
Deployment	Declares the desired pod template and replica count, and manages rolling updates by creating a new ReplicaSet and scaling the old one down
Service	A stable virtual IP and DNS name that load-balances across the pods matching its selector. Pods are ephemeral; Services are not
ConfigMap	Non-secret key/value configuration, injected as env vars or files
Secret	The same, for sensitive values. Base64-encoded, not encrypted at rest by default
Namespace	A logical partition of the cluster for isolation and quota
HPA	HorizontalPodAutoscaler - adjusts the replica count based on observed metrics

6.2 Namespace

```
apiVersion: v1
kind: Namespace
metadata:
  name: cicd-demo
```

Isolates the workload from `kube-system`. Makes RBAC, quotas and a single-command cleanup possible.

6.3 ConfigMap

```
data:
  APP_ENVIRONMENT: "production"
  APP_RELEASE_MESSAGE: "Release 2 - this change reached Azure automatically"
  APP_IMAGE_TAG: "${IMAGE_TAG}"        # injected by the pipeline
  APP_BUILD_NUMBER: "${BUILD_NUMBER}"
  APP_COMMIT_SHA: "${COMMIT_SHA}"
```

The build-provenance keys are what let the running application display which build it is - that is how the browser demonstration proves a redeployment happened. Real credentials would never go here; they belong in a Secret backed by Azure Key Vault.

6.4 Deployment - field by field

```
spec:
  replicas: 2
  revisionHistoryLimit: 5
  selector:
    matchLabels:
      app.kubernetes.io/name: cicd-demo-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

Field	Meaning and why this value
replicas: 2	Two pods, so one can be replaced while the other serves. With 1 replica and maxUnavailable 0 a rolling update cannot proceed
revisionHistoryLimit: 5	Keep 5 old ReplicaSets for rollback; more would waste etcd space
selector.matchLabels	How the Deployment finds its pods. Immutable after creation - changing it requires deleting the Deployment
strategy: RollingUpdate	Replace pods gradually. The alternative, Recreate, kills all pods first: simpler but causes downtime
maxSurge: 1	At most one extra pod above the desired count during an update
maxUnavailable: 0	Never drop below the desired count. This is what makes the update zero-downtime - a new pod must be Ready before an old one is removed

Probes - three of them, and they are not interchangeable

```
startupProbe:    /actuator/health/readiness  periodSeconds 5   failureThreshold 30
readinessProbe:  /actuator/health/readiness  periodSeconds 10  failureThreshold 3
livenessProbe:   /actuator/health/liveness   periodSeconds 20  failureThreshold 3
```

Probe	Question it answers	What happens on failure
startupProbe	Has the app finished booting?	Keeps restarting the clock. While it runs, the other two probes are disabled. 5 s x 30 = up to 150 s for the JVM to warm up
readinessProbe	Can it serve traffic right now?	The pod is removed from the Service's endpoints. It is NOT restarted - useful for a temporary dependency outage
livenessProbe	Is it alive, or wedged?	The container is restarted

Why a startupProbe at all? Without one you must set `initialDelaySeconds` on the liveness probe large enough for the slowest possible start, which delays detection of a genuine hang for the whole life of the pod. A startupProbe lets you be patient at boot and aggressive afterwards.

Resources - requests versus limits

```
resources:
  requests: { cpu: "150m", memory: "320Mi" }
  limits:   { cpu: "500m", memory: "640Mi" }
```

requests is what the scheduler reserves - it decides which node the pod fits on, and it is guaranteed. **limits** is the ceiling. Exceeding the CPU limit causes throttling; exceeding the memory limit causes the container to be **OOM-killed**. `150m` means 150 millicores, i.e. 0.15 of a core. `Mi` is mebibytes (1024-based); `M` would be megabytes (1000-based).

These numbers are deliberately small: one `Standard_B2s` node has 2 vCPU and 4 GiB, and the system pods need part of it. 2 x 150m leaves room. If pods go `Pending` with "Insufficient cpu", this is why.

securityContext - every field, and what it stops

```
# pod level
runAsNonRoot: true
runAsUser: 10001
runAsGroup: 10001
fsGroup: 10001
seccompProfile: { type: RuntimeDefault }
# container level
allowPrivilegeEscalation: false
readOnlyRootFilesystem: true
capabilities: { drop: ["ALL"] }
```

Field	What it prevents
runAsNonRoot / runAsUser 10001	A container escape lands as an unprivileged user, not root
allowPrivilegeEscalation: false	Blocks setuid binaries from gaining privileges inside the container
readOnlyRootFilesystem: true	An attacker cannot write a payload to disk. Requires an emptyDir at /tmp, because embedded Tomcat needs a writable temp directory
capabilities drop ALL	Removes Linux capabilities such as NET_RAW and CHOWN that the app never uses
seccompProfile RuntimeDefault	Restricts which syscalls the container may make
fsGroup	Mounted volumes are group-owned by 10001 so the non-root user can write to /tmp

6.5 Service - and the four types

```
spec:
  type: LoadBalancer
  selector:
    app.kubernetes.io/name: cicd-demo-app
  ports:
    - name: http
      port: 80          # the Service's port
      targetPort: http  # the container's named port (8080)
```

Type	Behaviour
ClusterIP	Default. Reachable only inside the cluster, at a virtual IP and a DNS name
NodePort	Opens the same high port (30000-32767) on every node, which forwards to the Service
LoadBalancer	Asks the cloud provider for a real load balancer and public IP. On AKS this creates an Azure Standard Load Balancer. Builds on NodePort underneath
ExternalName	A DNS CNAME to an external hostname; no proxying

Service DNS. Inside the cluster the full name is `cicd-demo-app.cicd-demo.svc.cluster.local`, i.e. `<service>.<namespace>.svc.cluster.local`. From inside the same namespace, just `cicd-demo-app` resolves.

6.6 HorizontalPodAutoscaler

```
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: cicd-demo-app }
  minReplicas: 2
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 70 }
```

Scales between 2 and 5 pods to keep average CPU near 70 percent of the **request** (not the limit). It needs `metrics-server`, which AKS installs by default. Note the interaction: the HPA owns the replica count once it is active, so editing `replicas` by hand is temporary - the HPA will correct it.

6.7 How the manifests get this build's image

The manifests are templates containing `${IMAGE}`, `${IMAGE_TAG}`, `${BUILD_NUMBER}` and `${COMMIT_SHA}`. `k8s/render.sh` substitutes them with `envsubst`, passing an explicit list of variable names so nothing else in the YAML can be touched, and writes the result to `k8s/rendered/`, which the pipeline archives.

Why not Helm or Kustomize? Both are better at scale. For five manifests whose only variable parts are an image reference and four strings, `envsubst` is transparent - you can read the rendered file and see exactly what was applied - and adds no tool to install or release lifecycle to explain. Say that plainly if asked; it is a defensible trade-off rather than ignorance of Helm.

Part 7 - Live Operations Recipes

These are the "change something right now" questions. Each recipe gives the command, what actually happens, and the thing worth saying while you type.

First, always: point kubectl at the right cluster. `kubectl config get-contexts` shows which is current; `az aks get-credentials -g rg-cicd-cia2 -n aks-cicd-cia2 --admin --overwrite-existing` selects AKS.

7.1 Add one more pod (scale to 3)

Three ways. Know all three and when each is right.

(a) Imperative - instant, not persisted

```
kubectl -n cicd-demo scale deployment/cicd-demo-app --replicas=3
kubectl -n cicd-demo get pods -w
```

The Deployment creates one more pod from the existing ReplicaSet. Fast and ideal for a live demo. **But** it is not in Git, so the next pipeline run re-applies `replicas: 2` and the extra pod disappears. Say that out loud - it shows you understand GitOps drift.

(b) Declarative - the correct way

```
# edit k8s/02-deployment.yaml:  replicas: 2  ->  replicas: 3
git commit -am "Scale to 3 replicas"
git push
# polling triggers the pipeline, which applies it to AKS
```

Slower, but the cluster matches the repository and the change is reviewed and auditable.

(c) Let the HPA do it

```
kubectl -n cicd-demo get hpa
kubectl -n cicd-demo describe hpa cicd-demo-app
```

The HPA already scales 2-5 on 70 percent CPU. If it is active, your manual `scale` will be overridden within a minute or two.

On a single `Standard_B2s` node, a third pod requesting 150m CPU may not fit alongside the system pods. If it stays `Pending`, run `kubectl -n cicd-demo describe pod <name>` and read the Events - it will say "Insufficient cpu". Fix by adding a node (7.8) or lowering the request.

7.2 Change the pod spec

"Pod spec" means anything under `spec.template.spec` in the Deployment. Any change there creates a **new ReplicaSet** and triggers a rolling update - that is the mechanism, and it is worth stating.

Live edit, for a demo

```
kubectl -n cicd-demo edit deployment/cicd-demo-app
# opens the live object in an editor; save and the rollout starts immediately
kubectl -n cicd-demo rollout status deployment/cicd-demo-app
```

Targeted change with patch

```
# change the memory limit
kubectl -n cicd-demo patch deployment/cicd-demo-app --type=json -p '[{
  "op":    "replace",
  "path":  "/spec/template/spec/containers/0/resources/limits/memory",
  "value": "768Mi"
}]'
```

Add or change an environment variable

```
kubectl -n cicd-demo set env deployment/cicd-demo-app APP_ENVIRONMENT=staging
```

Note the subtlety: the app reads most settings from the ConfigMap via `envFrom`. Editing the ConfigMap alone does **not** restart the pods, so the change is not picked up until they restart:

```
kubectl -n cicd-demo edit configmap cicd-demo-app-config
kubectl -n cicd-demo rollout restart deployment/cicd-demo-app
```

That is a classic question: "you changed the ConfigMap, why did nothing happen?" Because environment variables are injected at container start and never re-read. A ConfigMap mounted as a *volume* does update in place, but the application still has to notice.

7.3 Change the image

```
kubectl -n cicd-demo set image deployment/cicd-demo-app \
    app=acrcicdcia2.azurecr.io/cicd-demo-app:7-abc1234
kubectl -n cicd-demo rollout status deployment/cicd-demo-app
```

`app` is the container name, not the image name. `kubectl set image <resource> <container>=<image>`.

7.4 Roll back

```
kubectl -n cicd-demo rollout history deployment/cicd-demo-app
kubectl -n cicd-demo rollout undo deployment/cicd-demo-app
kubectl -n cicd-demo rollout undo deployment/cicd-demo-app --to-revision=2
```

This is why `revisionHistoryLimit: 5` matters, and why the Deployment sets the `kubernetes.io/change-cause` annotation: the history shows "Jenkins build 8 from commit 95e0d05" per revision rather than a meaningless list of numbers.

Rollback is fast because the old image is still in ACR. Immutable tags per build are what make that possible - with `latest` only, there is nothing to roll back *to*.

7.5 Pause and resume a rollout

```
kubectl -n cicd-demo rollout pause deployment/cicd-demo-app
kubectl -n cicd-demo rollout resume deployment/cicd-demo-app
```

Pausing lets you batch several spec changes into one rollout instead of triggering one per edit.

7.6 Debug a pod that will not start

```
kubectl -n cicd-demo get pods
kubectl -n cicd-demo describe pod <pod>          # Events at the bottom - read these first
kubectl -n cicd-demo logs <pod>
kubectl -n cicd-demo logs <pod> --previous       # logs from the crashed instance
kubectl -n cicd-demo exec -it <pod> -- sh        # shell inside (read-only FS, remember)
kubectl -n cicd-demo get events --sort-by=.lastTimestamp
```

Status	Usual cause	What to check
ImagePullBackOff	Cannot pull the image	AcrPull missing: az aks update --attach-acr; or a typo in the image name
CrashLoopBackOff	Container starts then exits	kubectl logs --previous; usually a config or startup error
Pending	Cannot be scheduled	describe pod -> Events. Usually Insufficient cpu/memory
CreateContainerConfigError	Missing ConfigMap or Secret key	Check the name referenced by envFrom exists
0/1 Running	Readiness probe failing	describe pod; curl the probe path from inside

7.7 Expose it differently

```
# temporary access without touching the Service
```

```
kubectl -n cicd-demo port-forward svc/cicd-demo-app 8081:80
# change the Service type
kubectl -n cicd-demo patch svc cicd-demo-app -p '{"spec":{"type":"NodePort"}}'
```

`port-forward` targets the Service, so it survives a rolling update. It is per-user and local - not a substitute for a LoadBalancer.

7.8 Add a node / resize the cluster

```
az aks scale -g rg-cicd-cia2 -n aks-cicd-cia2 --node-count 2
az aks nodepool list -g rg-cicd-cia2 --cluster-name aks-cicd-cia2 -o table
kubectl get nodes -o wide
```

Doubles the node cost. Distinguish clearly: **scaling pods** is `kubectl scale` / the HPA and is free if capacity exists; **scaling nodes** is `az aks scale` / the cluster autoscaler and costs money.

7.9 Node maintenance

```
kubectl cordon <node>                              # stop new pods landing here
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
kubectl uncordon <node>
```

7.10 Inspect the live configuration

```
kubectl -n cicd-demo get deployment cicd-demo-app -o yaml
kubectl -n cicd-demo get deployment cicd-demo-app \
    -o jsonpath='{.spec.template.spec.containers[0].image}'
kubectl -n cicd-demo get svc cicd-demo-app \
    -o jsonpath='{.status.loadBalancer.ingress[0].ip}'
kubectl -n cicd-demo top pods        # needs metrics-server
```

7.11 Add a second container to the pod (sidecar)

If asked to add a sidecar, the point is that containers in a pod share the network namespace and can share volumes, so they talk over `localhost`:

```
spec:
  template:
    spec:
      containers:
        - name: app
          image: ...
        - name: log-shipper          # second container
          image: busybox:1.36
          command: ["sh","-c","tail -f /var/log/app/app.log"]
          volumeMounts:
            - name: logs
              mountPath: /var/log/app
```

7.12 Trigger the pipeline yourself

```
# in the Jenkins UI: Build with Parameters -> choose DEPLOY_TARGET -> Build
# or from a terminal:
bash infra/jenkins-bootstrap.sh --build
DEPLOY_TARGET=local RUN_SONAR=false bash infra/jenkins-bootstrap.sh --build
```

Part 8 - Security

8.1 What this implementation actually does

1. **Secret detection as a hard gate.** Gitleaks scans the full history on every build and fails it on any finding, with six custom rules for the credential types this project handles.
2. **No credentials in Git.** `.gitignore` and `.dockerignore` both exclude `.env`, `*.pem`, `*.key`, `kubeconfig*` and `*-credentials.json`. Jenkins holds the four secrets and injects them only inside `withCredentials`, which masks them in logs.
3. **Least privilege in Azure.** The service principal is scoped to one resource group, with AcrPush on the registry and Cluster Admin on the cluster - nothing subscription-wide. AKS pulls images with its own managed identity, so no registry password exists in the cluster.
4. **Hardened container.** JRE-only image, non-root UID 10001, read-only root filesystem, no privilege escalation, all capabilities dropped, RuntimeDefault seccomp profile.
5. **Static analysis as a gate.** The Quality Gate blocks the build on new vulnerabilities and security hotspots.
6. **Provenance.** Every image carries OCI labels and env vars naming its commit and build, so anything running can be traced to source.
7. **Supply-chain pinning.** Base images pinned to specific tags, Maven pinned by the wrapper, Gitleaks and kubectl pinned by version in the Dockerfile.

8.2 Limitations - state these before you are asked

Volunteering limitations reads as competence. Being caught without them does not.

#	Limitation	Production remedy
1	Jenkins on plain HTTP with a single local admin account	TLS in front of it; an identity provider instead of a local account
2	The application endpoint is HTTP only	Ingress controller with TLS, or Azure Front Door
3	The Jenkins container runs as root and mounts the host Docker socket - root on the socket is root on the host	Rootless BuildKit, Buildah, or az acr build (server-side, no local daemon)
4	A long-lived service principal secret is stored in Jenkins	OIDC workload-identity federation - no stored secret at all
5	AKS accessed with --admin local-account credentials, bypassing Entra ID RBAC	Disable local accounts, use Entra ID with kubelogin, bind a narrow Role
6	No container image vulnerability scan	A Trivy stage failing on HIGH/CRITICAL
7	No image signing or SBOM	Cosign signatures and a Syft SBOM, enforced by admission policy
8	Single node, single Jenkins host - no high availability	Multi-node pool across availability zones; Jenkins with agents
9	No NetworkPolicies - pods can talk to anything	Azure or Calico network policy, default-deny egress
10	No staging environment or approval gate; main goes straight to the only environment	Separate namespaces or clusters with a manual approval before production
11	Kubernetes Secrets are base64, not encrypted	Azure Key Vault with the Secrets Store CSI driver

Part 9 - Problems Actually Hit

These are real failures from building this, with their root causes. They make excellent "challenges encountered" material because they are specific and the diagnosis is the interesting part.

Symptom	Root cause	Fix / lesson
kubectl apply said "unchanged" and no new code deployed	The manifest referenced a mutable latest tag, so the pod template was byte-identical	Tag every image -. The lesson: a green pipeline is not evidence of a deployment
Pipeline green but the app was broken	The pipeline verified that the API call succeeded, not the outcome	Added stage 8: rollout status plus an assertion that the live endpoint reports this build's tag
Container failed at startup with a read-only filesystem	Embedded Tomcat needs a writable temp directory	Mount an emptyDir at /tmp and keep the rest read-only
Jenkins could not run docker build	On Docker Desktop the socket is root:root mode 0660, and the jenkins user is neither	Run the container as root; recorded as limitation 3 rather than hidden
ACR creation failed claiming the name was taken	The real error was MissingSubscriptionRegistration - providers unregistered on a new subscription. The script had suppressed stderr with 2>/dev/null and guessed	Register providers; never replace a provider's error with a guess. A wrong error message sends you looking in the wrong place
Role assignment failed with (MissingSubscription)	Git Bash rewrote the scope /subscriptions/... into C:/Program Files/Git/subscriptions/...	export MSYS_NO_PATHCONV=1 and MSYS2_ARG_CONV_EXCL='*'
A script died claiming Python was missing, after a 10-minute wait	command -v python3 succeeds on Windows even when python3 is the Microsoft Store alias stub that fails when run	Test that the interpreter actually executes, not merely that it resolves
Image build failed on curl: (6) Could not resolve host	Transient DNS, right after other hosts had resolved fine	--retry-all-errors; plain --retry does not retry a resolution failure
Jenkins container would not start: port 8090/8080 in use	A separately installed native Jenkins already held 8080	Publish on a different host port; identify the holder with Get-NetTCPConnection
Scripts failed from PowerShell with execvpe(/bin/bash)	In PowerShell, bash resolves to the WSL shim, and the only distro had no /bin/bash	Run shell scripts from Git Bash
Shell scripts failed on Linux with bad interpreter	Git on Windows checks files out with CRLF	.gitattributes forcing eol=lf for *.sh, mvnw, Jenkinsfile and YAML
SonarCloud rejected the pipeline's analysis	Automatic Analysis was enabled and is mutually exclusive with CI analysis	Turn it off in Administration -> Analysis Method
First cost estimate was far too low	The invisible costs were the Load Balancer and the default 128 GB Premium SSD disks, which bill even while compute is stopped	Move Jenkins off Azure, 32 GB StandardSSD disk, pause between sessions

The meta-lesson, worth saying in a viva. Most of these were not coding errors. They were *diagnostic* errors: an error message that guessed instead of reporting, a check that tested existence instead of function, a pipeline that verified a call instead of an outcome. The fix in each case was to make the system tell the truth about itself.

Part 10 - Question Bank

Concepts

Q. What is the difference between Continuous Integration, Continuous Delivery and Continuous Deployment?

CI is merging and automatically building and testing every change, so integration problems surface immediately. Continuous Delivery means every change that passes is *releasable* - the artifact is built and verified, but a human decides when to push it. Continuous Deployment goes further and releases automatically with no human gate. This project is Continuous Deployment on main: a passing build deploys to AKS unattended.

Q. What is a pipeline "gate"?

A check that can stop the pipeline. Gitleaks and the SonarQube Quality Gate are gates: they fail the build. A report that is merely archived is not a gate. The distinction matters because only a gate changes behaviour.

Q. Why containers rather than deploying a JAR to a VM?

The image bundles the JRE and the application, so there is no "which Java is on the server?" question. Images are immutable and versioned, so rollback is pulling an older tag. And the same artifact runs on a laptop and in the cloud.

Q. Why Kubernetes rather than just running the container?

Declarative desired state, self-healing, health-gated rolling updates, horizontal scaling and a stable Service abstraction in front of ephemeral pods. Running a bare container gives you none of that.

Q. What is declarative versus imperative?

Imperative says what to do (`kubectl scale --replicas=3`); declarative says what should be true (`replicas: 3` in a file that is applied). Declarative is reproducible and reviewable, which is why the manifests are in Git and the imperative commands are only for demos.

Jenkins and triggering

Q. When exactly does your pipeline run?

Three ways. SCM polling, `pollSCM('H/2 * * * *')`, is the primary automatic trigger: Jenkins asks GitHub every two minutes whether main has moved and builds if it has. `githubPush()` is also declared and would fire instantly on a push, but GitHub cannot reach localhost, so it only works behind a tunnel. And a human or the REST API can start one manually.

Q. What does the H in H/2 mean?

Hash. Jenkins hashes the job name to choose an offset within the interval, so jobs do not all poll at the same instant. `*/2` would synchronise every job on the controller.

Q. Polling is wasteful - why not a webhook?

A webhook is better and is declared in the Jenkinsfile. It needs an inbound connection, and Jenkins here runs on a laptop with no public address. Polling is outbound, so it works regardless. The trade-off is up to two minutes of latency.

Q. What is a declarative Jenkinsfile and why keep it in the repository?

A structured pipeline definition (`pipeline { stages { ... } }`) as opposed to free-form scripted Groovy. Keeping it in the repository means the pipeline is version-controlled with the code, reviewed like code, and a branch can change its own build.

Q. How are secrets kept out of the console log?

`withCredentials` binds them to environment variables for the block and masks matching strings in the output. The log shows `-Dsonar.token=****`.

Q. Why disableConcurrentBuilds?

Two builds deploying to the same namespace simultaneously would race: whichever finished last would win, possibly deploying the older commit.

Docker

Q. What is a multi-stage build and what did it save?

Separate build and runtime stages, copying only the artifact forward. The published image is about 190 MB instead of about 600 MB, and it contains no compiler or build tool, which shrinks the attack surface.

Q. Why copy pom.xml and resolve dependencies before copying src?

Docker caches layers. Dependencies change rarely, source changes constantly. Resolving dependencies in their own earlier layer means a source-only change reuses the cache instead of re-downloading the repository.

Q. Why does the image run as UID 10001?

So a container escape lands as an unprivileged user. The Kubernetes securityContext enforces the same thing with `runAsNonRoot`, defence in depth.

Q. What is .dockerignore for?

It keeps files out of the build context: smaller and faster uploads, and - more importantly - it prevents `.env`, keys and kubeconfigs from accidentally being copied into an image layer.

Kubernetes

Q. Walk me through what happens when you change the image tag.

The Deployment's pod template changes, so the Deployment creates a new ReplicaSet and scales it up while scaling the old one down, respecting maxSurge 1 and maxUnavailable 0. Each new pod must pass its readiness probe before it receives traffic and before an old pod is removed, so there is no downtime. The old ReplicaSet is kept, scaled to zero, for rollback.

Q. Difference between a readiness and a liveness probe?

Readiness answers "can it serve traffic now?" - failure removes the pod from the Service endpoints but does not restart it. Liveness answers "is it wedged?" - failure restarts the container. Using liveness where you meant readiness causes restart loops during a transient dependency outage.

Q. What does a startupProbe add?

It gives a slow-starting application time to boot without making the liveness probe permanently lenient. While it runs, the other probes are suspended. Here 5 s x 30 allows up to 150 s for the JVM.

Q. requests versus limits?

requests is what the scheduler reserves and is guaranteed; limits is the ceiling. Exceeding the CPU limit throttles; exceeding the memory limit gets the container OOM-killed.

Q. Why maxUnavailable: 0?

It forbids dropping below the desired replica count during an update, so a new pod must be Ready before an old one goes. That is what makes the update zero-downtime.

Q. How does the Service find its pods?

By label selector - `app.kubernetes.io/name: cicd-demo-app` - not by pod name. Pods come and go; labels are stable.

Q. What do the Service types do?

ClusterIP is internal only. NodePort opens a high port on every node. LoadBalancer asks the cloud for a real load balancer and public IP, building on NodePort. ExternalName is a DNS alias.

Q. Who creates the public IP?

The Azure cloud-controller-manager, in response to `type: LoadBalancer`. It provisions an Azure Standard Load Balancer and a public IP in the AKS-managed `MC_*` resource group. It takes two to five minutes, which is why stage 8 polls for it.

Q. Why is there a second resource group starting with MC_?

AKS puts the cluster object in your group but the real compute, networking and load balancer in a managed cluster group it owns. You should not edit it by hand. Deleting your group removes both.

Q. You changed the ConfigMap and nothing happened. Why?

Environment variables are injected when the container starts and never re-read. You need `kubectl rollout restart deployment/cicd-demo-app`. A ConfigMap mounted as a volume does update in place, but the application must watch for it.

Q. How do you roll back?

`kubectl rollout undo deployment/cicd-demo-app`, optionally `--to-revision=N`. It works because old ReplicaSets are retained and the old images are still in ACR under immutable tags.

Q. A pod is Pending. What do you do?

`kubectl describe pod` and read the Events. On this one-node B2s cluster the usual answer is Insufficient cpu. Fix by adding a node or reducing the request.

Q. A pod is in ImagePullBackOff. Why?

The kubelet cannot pull the image. Most likely the AcrPull grant is missing - `az aks update --attach-acr acrcicdcia2` - or the image reference is wrong.

Q. What is the difference between scaling pods and scaling nodes?

`kubectl scale` or the HPA changes the number of pods and is free if the node has capacity. `az aks scale` changes the number of VMs and costs money. Pods cannot scale beyond what the nodes can hold.

Azure

Q. Why ACR rather than Docker Hub?

It is in the same cloud and region as the cluster, it is private, and AKS authenticates with its managed identity through the AcrPull role - so no registry password exists in the cluster or in Jenkins.

Q. What permissions does Jenkins have in Azure, and why those?

Contributor scoped to the single resource group, AcrPush on the registry so it can push but not administer it, and Azure Kubernetes Service Cluster Admin on the cluster so it can fetch credentials. Nothing subscription-wide.

Q. What does --attach-acr actually do?

Grants the cluster's kubelet managed identity the AcrPull role on that registry, so image pulls are authenticated without a secret.

Q. What does your infrastructure cost, and how do you control it?

About US$0.085/hour, roughly US$2.05/day: the B2s node, the Standard Load Balancer, a static public IP, a 32 GB StandardSSD disk and ACR Basic. The AKS control plane is free on the Free tier, and Jenkins runs locally so costs nothing. `azure-pause.sh` stops the cluster and drops it to about US$0.35/day; teardown deletes the resource group.

Quality and security

Q. Why Gitleaks when .gitignore already excludes secrets?

`.gitignore` is advisory and only helps for files nobody forces. It does nothing about a key pasted into a source file, and nothing about history. Gitleaks scans every commit and fails the build.

Q. A secret was committed and then deleted. Is it safe?

No. It is still in the git object database and recoverable by anyone who clones. It must be rotated, and the history rewritten if it is to be removed. This is exactly why the scan covers history rather than the working tree.

Q. What does the Quality Gate check?

On new code: no new blocker or critical issues, security hotspots reviewed, coverage at or above 80 percent, duplication below 3 percent. This project reports 95.7 percent instruction coverage.

Q. What would you add next, given more time?

A Trivy image scan failing on HIGH/CRITICAL; Cosign signing with an SBOM; a staging namespace with a manual approval before production; NetworkPolicies with default-deny egress; and replacing the service principal secret with OIDC workload-identity federation.

Appendix A - Command Cheat Sheet

Kubernetes

```
kubectl config get-contexts
az aks get-credentials -g rg-cicd-cia2 -n aks-cicd-cia2 --admin --overwrite-existing

kubectl -n cicd-demo get deployment,pods,svc -o wide
kubectl -n cicd-demo describe pod <pod>
kubectl -n cicd-demo logs <pod> [--previous] [-f]
kubectl -n cicd-demo exec -it <pod> -- sh
kubectl -n cicd-demo get events --sort-by=.lastTimestamp
kubectl -n cicd-demo top pods

kubectl -n cicd-demo scale deployment/cicd-demo-app --replicas=3
kubectl -n cicd-demo set image deployment/cicd-demo-app app=<registry>/<img>:<tag>
kubectl -n cicd-demo set env deployment/cicd-demo-app KEY=value
kubectl -n cicd-demo edit deployment/cicd-demo-app
kubectl -n cicd-demo rollout status   deployment/cicd-demo-app
kubectl -n cicd-demo rollout history  deployment/cicd-demo-app
kubectl -n cicd-demo rollout undo     deployment/cicd-demo-app [--to-revision=N]
kubectl -n cicd-demo rollout restart  deployment/cicd-demo-app
kubectl -n cicd-demo port-forward svc/cicd-demo-app 8081:80
```

Azure

```
az account show -o table
az resource list -o table
az group list -o table

az acr repository list --name acrcicdcia2 -o table
az acr repository show-tags --name acrcicdcia2 --repository cicd-demo-app \
    --orderby time_desc -o table

az aks show -g rg-cicd-cia2 -n aks-cicd-cia2 \
    --query "{state:provisioningState,power:powerState.code,k8s:kubernetesVersion}" -o json
az aks scale -g rg-cicd-cia2 -n aks-cicd-cia2 --node-count 2
az aks stop  -g rg-cicd-cia2 -n aks-cicd-cia2
az aks start -g rg-cicd-cia2 -n aks-cicd-cia2
az aks update -g rg-cicd-cia2 -n aks-cicd-cia2 --attach-acr acrcicdcia2

az role assignment list --assignee <appId> --all -o table
```

Local, Maven, Docker, Jenkins

```
./mvnw clean verify                # 29 tests + JaCoCo -> target/site/jacoco/index.html
./mvnw spring-boot:run             # run on :8080
docker build -t cicd-demo-app:local .
docker run --rm -p 9090:8080 cicd-demo-app:local

docker compose up -d --build       # start Jenkins on :8090
docker compose logs -f jenkins
docker compose exec jenkins cat /usr/local/share/toolchain.txt
docker compose stop | down [-v]

bash infra/azure-setup.sh          # provision
bash infra/local-connect.sh        # point Jenkins at Docker Desktop Kubernetes
bash infra/jenkins-bootstrap.sh --build
bash infra/azure-pause.sh | azure-resume.sh | azure-teardown.sh

gitleaks git . --config .gitleaks.toml --redact --exit-code 1
```

Appendix B - File Inventory

File	Purpose
Jenkinsfile	The 8-stage declarative pipeline, with DEPLOY_TARGET and RUN_SONAR parameters
docker-compose.yml	Runs the Jenkins controller locally on port 8090
jenkins/Dockerfile	Jenkins image with docker CLI, az, kubectl, gitleaks, ansible, envsubst
jenkins/plugins.txt	Plugins pre-installed at image build time
jenkins/init.groovy.d/01-security.groovy	Creates the admin user; skips the setup wizard
jenkins/job-config.xml	The job definition, applied through the REST API
pom.xml	Maven build, JaCoCo, SonarQube configuration
mvnw, .mvn/	Maven wrapper - pins Maven 3.9.9, so no Maven install is needed
Dockerfile	Multi-stage application image, runs as UID 10001
.dockerignore	Keeps the build context small and secret-free
.gitleaks.toml	Secret-scanning rules plus a narrow documented allowlist
.gitattributes	Forces LF endings so shell scripts run on Linux
.env.example	Template for the git-ignored .env (Jenkins admin credentials)
k8s/00..04-*.yaml	Namespace, ConfigMap, Deployment, Service, HPA
k8s/render.sh	Substitutes this build's image tag with envsubst
ansible/deploy-app.yml	USED by stage 7: renders manifests, applies, waits, reports
ansible/jenkins-setup.yml	OPTIONAL: provisions an Azure VM Jenkins instead
infra/azure-setup.sh	Creates resource group, providers, ACR, AKS, service principal
infra/azure-pause.sh / resume / teardown	Cost control and cleanup
infra/local-connect.sh	Points Jenkins at Docker Desktop Kubernetes
infra/jenkins-bootstrap.sh	Creates/updates the job and can start a build
infra/jenkins-azure-credentials.sh	Loads all four credentials into Jenkins
docs/SETUP.md / LOCAL.md / REPORT.md	Azure runbook, local runbook, technical report

Appendix C - Numbers To Memorise

Fact	Value
Pipeline stages	8
Unit tests	29, all passing
Coverage	95.7% instruction, 88.5% branch
Replicas	2 (HPA 2-5 at 70% CPU)
Image tag format	-, e.g. 8-95e0d05
Rolling update	maxSurge 1, maxUnavailable 0
Container user	UID 10001, non-root, read-only root filesystem

Fact	Value
Resource requests / limits	150m / 500m CPU, 320Mi / 640Mi memory
Polling interval	H/2 * * * * (about every 2 minutes)
Image size	about 190 MB (JRE Alpine) vs about 600 MB with the JDK
AKS node	1 x Standard_B2s (2 vCPU, 4 GiB), 32 GB StandardSSD, Free tier control plane
Cost	about US$2.05/day running, about US$0.35/day paused
Gitleaks rules	~170 default + 6 custom
Jenkins credentials	4: azure-sp, azure-tenant-id, azure-subscription-id, sonarcloud-token
Service	LoadBalancer, port 80 -> targetPort 8080

If you remember only one thing: every image is tagged `<build>-<commit>`, which is what makes rolling updates actually roll, rollback possible, and any running pod traceable to source. Almost every other design decision in this project follows from that one.